Kriptográfia és biztonság 7.

Hash + MAC

Hash függvények adat-struktúrákban

- a hash függvény egy kompressziós függvény
- tetszőleges input-hossz
- $H : \{0,1\}^* \mapsto \{0,1\}^n$, ahol $n = 128, 160, 256, etc$
- klasszikus alkalmazás: adat-struktúrák
  - elemek tárolása egy k méretű táblában
  - beszúrás és keresés konstans időben
  - az x elemet a $H(x)$ cellában tároljuk
  - x visszanyerhető $H(x)$ kiszámításával és a megfelelő cella-értékkel
- *ütközés*: $x \neq x' : H(x) = H(x')$
- „jó” hash függvény, ha kevés ütközés van
- megfelelően szétszórja az elemeket

Kriptográfiai hash függvények

- adat tömörítés
- kevés ütközés
- az ütközésmentesség
  - adat-struktúrákban csak elvárt
  - kriptóban elengedhetetlen
- adat-struktúrákban x és $H(.)$ független
- kriptóban a támadó tetszőlegesen választhatja x-et
- kripto hash-et sokkal nehezebb konstruálni...

Kriptográfiai hash függvények

Definíció

Egy ütközés a $H(.)$ függvényben egy $x \neq x'$ input-pár, amire $H(x) = H(x')$.
A $H(.)$ függvény ütközés-mentes, ha bármely PPT támadó csak elhanyagolható valószínűséggel talál ütközést.
A $H(.)$ függvény egy hash függvény, ha $H : \{0,1\}^* \mapsto \{0,1\}^n$.

Gyengébb biztonsági definíciók

1. Ütközés-mentesség: lásd fent
2. *Második őskép biztonság*: adott x-hez PPT támadó nem képes találni $x' \neq x : H(x') = H(x)$
3. *Őskép biztonság*: adott $y = H(x)$-hez, random (és ismeretlen) x esetén PPT támadó nem képes találni $x' : H(x') = y$ (magyarul egyirányú függvény)

Kriptográfiai hash függvények

Definíció

Egy ütközés a $H(.)$ függvényben egy $x \neq x'$ input-pár, amire $H(x) = H(x')$.
A $H(.)$ függvény ütközés-mentes, ha bármely PPT támadó csak elhanyagolható valószínűséggel talál ütközést.
A $H(.)$ függvény egy hash függvény, ha $H : \{0,1\}^* \mapsto \{0,1\}^n$.

Gyengébb biztonsági definíciók

1. Ütközés-mentesség: lásd fent
2. *Második őskép biztonság*: adott x-hez PPT támadó nem képes találni $x' \neq x : H(x') = H(x)$
3. *Őskép biztonság*: adott $y = H(x)$-hez, random (és ismeretlen) x esetén PPT támadó nem képes találni $x' : H(x') = y$ (magyarul egyirányú függvény)

Kriptográfiai hash függvények

Tervezési alapelvek

- Ütközés-mentesség
- Második őskép biztonság
- Őskép biztonság
- *Lavina hatás*: kis változás az inputban $\Rightarrow$ nagy változás az outputban
  - *Szigorú lavina kritérium:* ha egy input bitet megváltoztaunk $\Rightarrow$ minden output bit 1/2 valószínűséggel megváltozik
  - *Bit függetlenség kritérium:* $\forall i, j, k$: ha az i-ik input bit megváltozik $\Rightarrow$ a j, k output bitek megváltozása egymástól független

Támadások

Születésnap támadás

Egy $H : \{0,1\}^* \mapsto \{0,1\}^n$ hash-függvényre ütközés található 1/2 valószínűséggel, ha $2^{n/2}$ hash értéket kiszámítunk.

- Lényegesen gyorsabb, mint a brute-force
- $\Longrightarrow n \geq 160$
- Ha gyorsabban találunk ütközést, mint a születésnap támadás $\Longrightarrow$ a hash-t "feltörtük"

Merkle-Damgård transzformáció

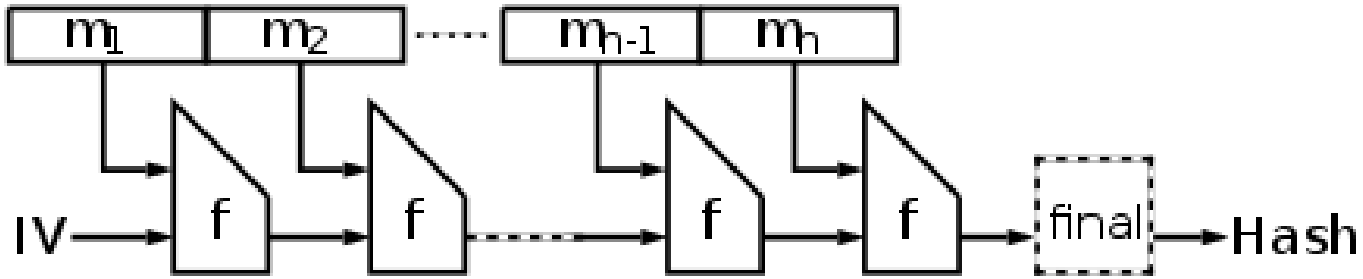
Alapötlet

- gyakorlati konstrukciók csak fix hosszú bemenetet kezelnek
- módszer tetszőleges inputú hash függvényekre
- legyen $h : \{0,1\}^{2n} \mapsto \{0,1\}^n$ fix bemenetú hash fv, $m \in \{0,1\}^*$, ahol $|m| = \ell < 2^n$
- láncolási szabályokhoz hasonló iteratív konstrukció
- $\Rightarrow H(.)$ egy tetszőleges inputú hash függvény

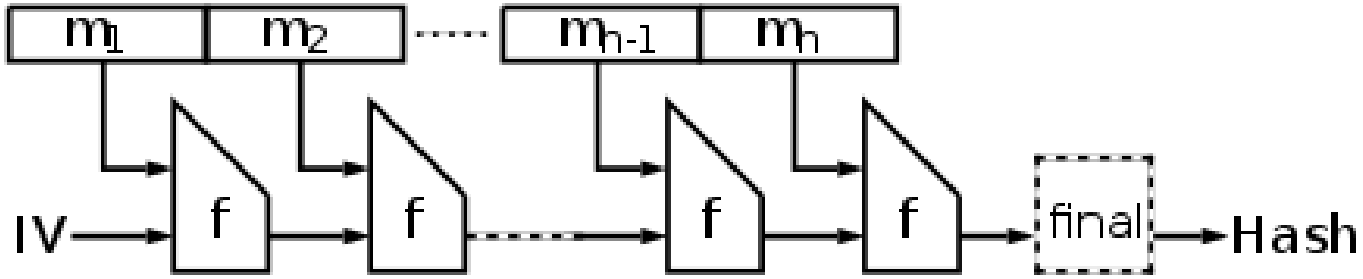
Merkle-Damgård transzformáció

Merkle-Damgård transzformáció

1. daraboljuk m-et n hosszú blokkokra: $b := \lceil \frac{\ell}{n} \rceil$ és $m = (m_1|m_2|\ldots|m_b)$
2. legyen $m_{b+1} := \ell \in \{0,1\}^k, z_0 := IV$
3. $i = 1, \ldots, b+1$ esetén számítsuk ki $z_i := h(z_{i-1}|m_i)$
4. $H(m) := z_{b+1}$



Merkle-Damgård transzformáció

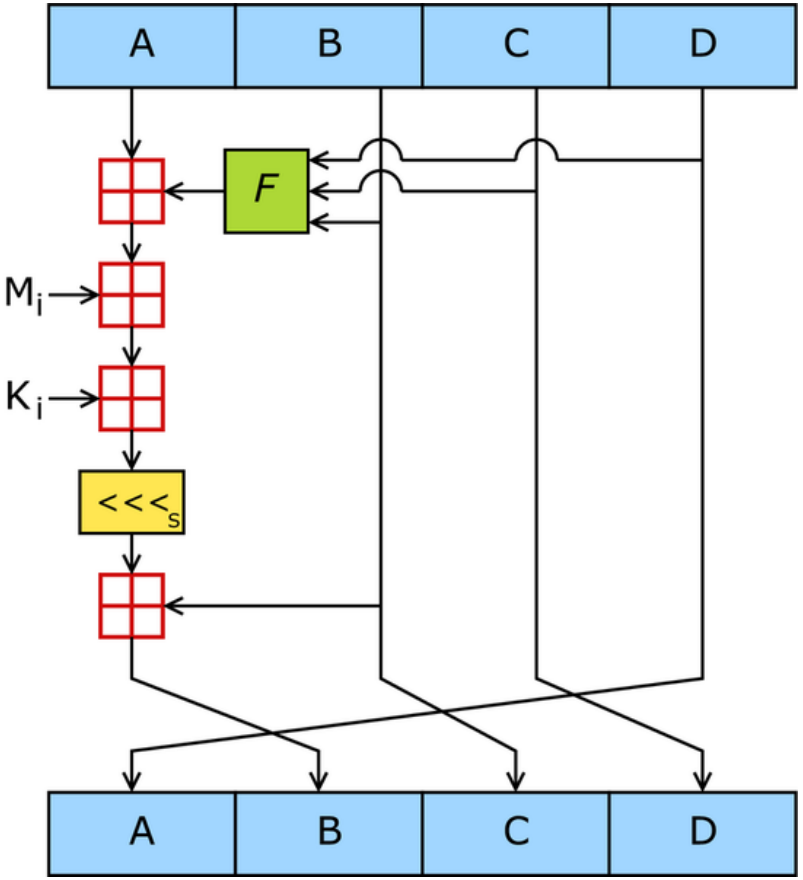


Tulajdonságok

- gyakorlatban: elég fix hosszúságot kezelő konstrukció
- elméletben: az összenyomás mennyisége mindegy
- inicializáló vektor – $IV : z_0$ szabadon választható
- biztonság:ha $h(.)$ ütközésmentes, akkor $H(.)$ is az

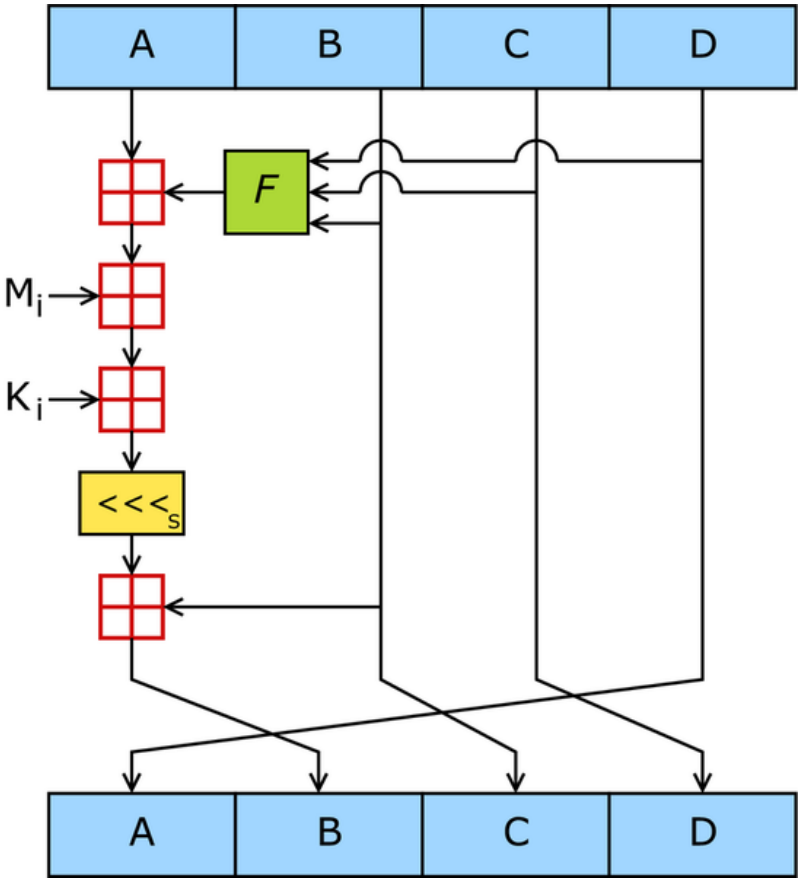
MD5 - leírás

- 512-ről 128 bitre összenyomás kiterjesztve Merkle-Damgård-dal
- 32-bites szavakon működik
- m felosztása 512(=16*32)-bit blokkokra
- 128(=4*32)-bit állapotokon operál
- A, B, C, D rögzített
- 4 kör, mindben 16 hasonló művelet
- négy nem-lineáris F:
  1. $F(B,C,D) = (B \wedge C) \vee (\neg B \wedge D)$
  2. $G(B,C,D) = (B \wedge D) \vee (C \wedge \neg D)$
  3. $H(B,C,D) = B \oplus C \oplus D$
  4. $I(B,C,D) = C \oplus (B \vee \neg D)$
- M_i az üzenet blokk
- K_i konstans, s eltolási paraméter (műveletenként változik)



MD5 – elemzés

- NEM ütközésmentes!
- 128 bit output $\Longrightarrow$ születésnap támadás...
- 1992 - MD5 publikálása
- 1993 - „pszeudo-ütközés” a kompressziós függvényben (IV alapú támadás)
- 1996 - ütközés a kompressziós függvényben
- 2004 - MD5CRK, születésnap alapú elosztott támadás
- 2004 - hash ütközés 1 órán belül (analitikus támadás)
- 2005 - gyakorlati ütközés két X.509 tanúsítványban, aminek különböző a kulcsa, de uaz az MD5 hash-e
- 2010 - első publikált egy blokkos ütközés



SHA hash család

- SHA – Secure Hash Algorithm
- U.S. NSA fejlesztés, U.S. NIST publikálta

SHA-0 (1993)

- 160-bit output, 32-bites szavak, 80 kör
- műveletek: $\oplus, \boxplus, \wedge, \vee, \lll$
- ütközés...

SHA-1 (1995)

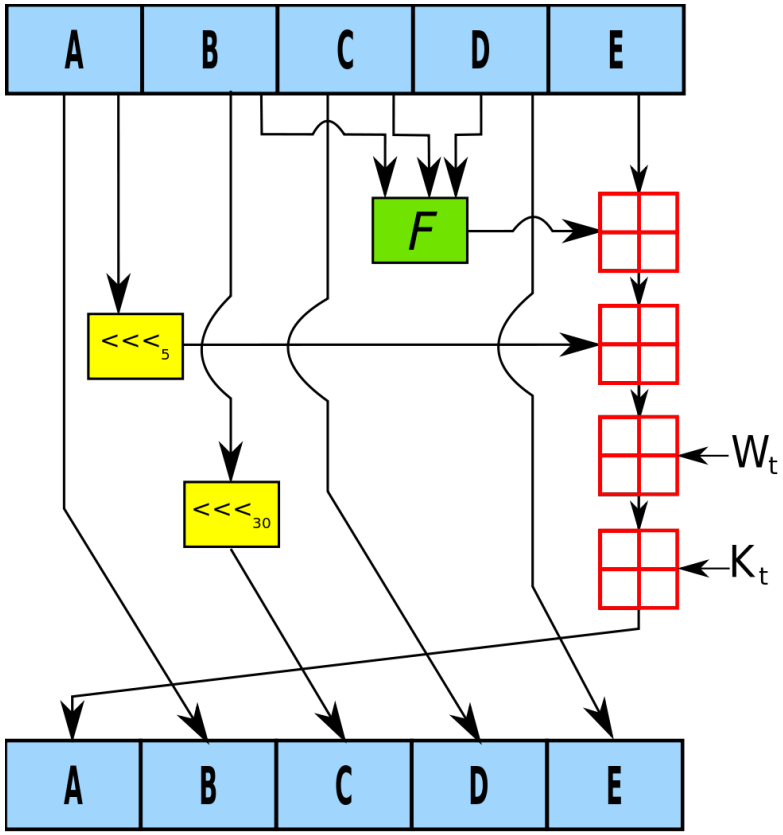
- 160-bit output, 32-bites szavak, 80 kör
- ellenállóbb, elméleti támadás 2^{61} lépésben (2011)

SHA-2 (2001) = SHA-256/SHA-512

- 256/512-bit output, 32/64-bites szavak, 64/80 kör
- nincs (ismert) ütközés

SHA-3 (2014-)

- eltérő design
- az SHA-2 alternatívája



SHA-1, original diagram for Wikipedia created by User:Matt Crypto

NIST hash függvény verseny (2007 – 2012)

Az AES versenyhez hasonló fejlesztési folyamat

Oct. 2008 beküldési határidő

Dec. 2008 Round 1: 51 megmaradt jelölt

Feb. 2009 NIST konferencia: beküldött algoritmusok ismertetése

Jul. 2009 Round 2: 14 elfogadott jelölt

Aug. 2010 CRYPTO 2010: a második körös jelöltek elemzése

Dec. 2010 Döntősök kihirdetése

- teljesítmény: alacsony hardver követelmények
- biztonság: lehetséges kripto/tervezési gyengeségek
- elemzés: kriptanalízis (hiánya) a teljes kripto-közösséggel
- diverzitás: különböző működési módok és belső állapotok

Dec. 2012 Győztes: Keccak

Aug. 2013 NIST változásokat jelent be a javasolt standardhoz képest, a "jobb biztonság/teljesítmény arány érdekében"...

Aug. 2015 Keccak az új SHA-3 hash standard

SHA-3/Keccak

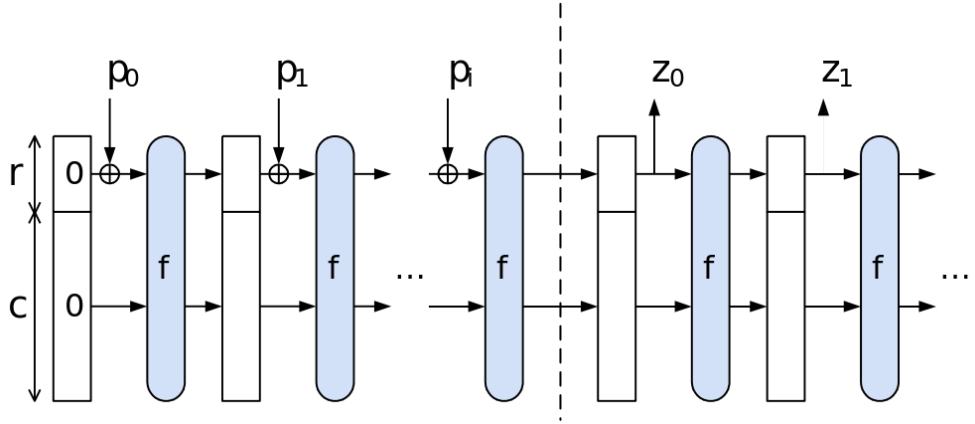


Diagram of a sponge construction from http://sponge.noekeon.org/

- Bertoni, Daemen, Peteers és Van Assche fejlesztése
- *Szivacs konstrukció* – egy rögzített hosszúságú f permutáció + padding szabály:

1. m pad-elve és felszabdalva r-bites p_i blokkokra
2. felszívás: iterált f függvény végrehajtás: p_i XOR-olva a hash állapotba adott r arányban összefésülve az előző f kimenettel
3. összenyomás: z_i output blokkok kinyerése hasonlóan, ugyanazzal az aránnyal

Biztonság vs. Integritás

- biztonságos kommunikáció
  - Aliz üzenetet küld Bob-nak
  - nyílt kommunikációs csatornán
  - biztonság
  - **eszköz:** titkosítás
- integritás
  - Aliz üzenetet küld Bob-nak
  - nyílt kommunikációs csatornán
  - autenticitás (hívó-ID, email cím)
  - integritás
  - bármely nem észlelt üzenet-módosítást megakadályoz
  - ellenséges módosítás nem kritpo feladat (fizikai ellenintézkedések)
  - **eszköz:** ???

Biztonság vs. Integritás

- biztonságos kommunikáció
  - Aliz üzenetet küld Bob-nak
  - nyílt kommunikációs csatornán
  - biztonság
  - **eszköz:** titkosítás
- integritás
  - Aliz üzenetet küld Bob-nak
  - nyílt kommunikációs csatornán
  - autenticitás (hívó-ID, email cím)
  - integritás
  - bármely nem észlelt üzenet-módosítást megakadályoz
  - ellenséges módosítás nem kritpo feladat (fizikai ellenintézkedések)
  - **eszköz:** ???

Titkosítás vs. üzenet autentikáció

- Titkosítás folyamtitkosítóval
  - legyen $c := E_k(m) = G(k) \oplus m$ a titkos szöveg, ahol $G(.)$ egy PRG
  - egy bit megváltoztatása c-ben $\Longrightarrow$ uaz a bit megváltozik m-ben
  - Pl: a 11-ik lsb megváltoztatása 1000$ eltérést okoz...
  - a rendszer ettől még biztonságos...
  - hasonló támadás a one-time pad ellen is...
- Titkosítás blokktitkosítóval
  - uez a támadás OTR és CTR módban
  - kicsit komplikáltabb támadás ECB és CBC módban
- titkosítás önmagában nem garantál integritást
  - c teljesen elrejti m-et
  - DE a támadó módosíthatja c-t értelmesen!
  - bármely lehetséges c megfelel egy m-nek...
- valami új kell

Titkosítás vs. üzenet autentikáció

- Titkosítás folyamtitkosítóval
  - legyen $c := E_k(m) = G(k) \oplus m$ a titkos szöveg, ahol $G(.)$ egy PRG
  - egy bit megváltoztatása c-ben $\Longrightarrow$ uaz a bit megváltozik m-ben
  - Pl: a 11-ik lsb megváltoztatása 1000$ eltérést okoz...
  - a rendszer ettől még biztonságos...
  - hasonló támadás a one-time pad ellen is...
- Titkosítás blokktitkosítóval
  - uez a támadás OTR és CTR módban
  - kicsit komplikáltabb támadás ECB és CBC módban
- titkosítás önmagában nem garantál integritást
  - c teljesen elrejti m-et
  - DE a támadó módosíthatja c-t értelmesen!
  - bármely lehetséges c megfelel egy m-nek...
- valami új kell

Üzenet autentikációs kód: definíció

Probléma

- a kommunikáló feleknek van egy közös titkuk (a titkos kulcs)
- autentikált üzenet küldése
- az üzenet módosították-e?
- Message Authentication Code (MAC)

Üzenet autentikációs kód: definíció

Definíció

Egy üzenet autentikációs kód egy $(Gen, Mac, Vrfy)$ *hármas, amire:*

- Gen *a kulcsgenerálás, ami adott* 1^n *biztonsági paraméterre egy* k *titkos kulcsot ad vissza, amire* $|k| \geq n$
- Mac *a címkegenerálás, ami* k *kulcs és* $m \in \{0,1\}^*$ *üzenet esetén egy* t *MAC címkét ad vissza:* $t := Mac_k(m)$
- $Vrfy$ *az ellenőrzés, ami* k *kulcs,* t *címke és* m *üzenet esetén egy* $b := Vrfy_k(m,t)$ *bitet ad visssza, ami* $b = 1$*, ha a* t *érvényes MAC címke* m*-hez és 0 egyébként*

Továbbá a rendszernek korrektnek kell lennie: tetszőleges paraméterekre

$$Vrfy_k(m, Mac_k(m)) = 1.$$

Üzenet autentikációs kód: biztonság definíció

Hogyan támadjuk ezt a rendszert?

- a támadó az alábbi lépéseket végzi:
  1. valamely üzenetekhez tartozó MAC címkéket kérdez Aliztól (hatás az m tartalmára)
  2. az eredmények alaján számolgat
  3. kiad egy *hamisítványt*: egy érvényes t címkét valamely új m-hez (korábban nem kérdezett)
- biztonságos, ha ez a támadás „nehéz”

Definíció

Egy autentikációs kód adaptív választott nyíltszöveges támadás ellen biztonságos (vagy röviden biztonságos), ha minden PPT támadó egy m üzenetre egy érvényes t MAC címkét csak elhanyagolható valószínűséggel tud készíteni több t' címke lekérdezése után $m' \neq m$ üzenetekre.

Üzenet autentikációs kód: biztonság definíció

Definíció

Egy autentikációs kód adaptív választott nyíltszöveges támadás ellen biztonságos (vagy röviden biztonságos), ha minden PPT támadó egy m üzenetre egy érvényes t MAC címkét csak elhanyagolható valószínűséggel tud készíteni több t' címke lekérdezése után $m' \neq m$ üzenetekre.

- túl erős definíció?
  - támadó bármely üzenet címkéjét lekérdezheti
  - bármely üzenethez tartozó címke generálásával „feltörik” a rendszert
- gyakorlatban csak az értelmes üzenetek érdekesek
- mit jelent az ”értelmes„?
- *Visszajátszásos támadás*
  - egyéb megoldások: sorozatszámok vagy idő-pecsétek az $m$hez fűzve
  - hátrányok: tárolási vagy szinkronizálási problémák

MAC konstrukció rögzített hosszú üzenetre

Rögzített hosszú MAC

Legyen $PRF : \{0,1\}^n \mapsto \{0,1\}^n$ pszeudovéletlen függvény. Ekkor az alábbi egy rögzített hosszú MAC:

Gen: $k \in_R \{0,1\}^n$

Mac: adott k kulcs és $m \in \{0,1\}^n$ üzenetre a címke $t := PRF_k(m)$

$Vrfy$: adott k kulcsra, $m \in \{0,1\}^n$ üzenetre és $t \in \{0,1\}^n$ címkére az output 1 $\Leftrightarrow t = PRF_k(m)$

- ha PRF pszeudovéletlen függvény $\Rightarrow$ biztonságos
- hátrány: csak fix hosszúságú m-re
- randomizálással (+ pár trükkel) kiterjeszthető tetsz. hosszra

MAC hash függvényekből: HMAC

HMAC

legyen $h : \{0,1\}^{2n} \mapsto \{0,1\}^n$ kompressziós fv és $H : \{0,1\}^* \mapsto \{0,1\}^n$ az ebből gyártott Merkle-Damgård hash fv és legyen $IV, ipad, opad \in \{0,1\}^n$ rögzített

Gen: $k \in_R \{0,1\}^n$

Mac: adott k kulcsra és $m \in \{0,1\}^*$ üzenetre a címke
$t := h(h(IV|k \oplus opad)|H_{IV}(k \oplus ipad|m))$

$Vrfy$: az output 1 $\Leftrightarrow t = Mac_k(m)$

- ha h ütközésmentes, akkor biztonságos MAC
- hossz-alapú támadások ellen véd